

EVE310 Lab 2

September 4, 2025

Announcements

- Use the modules feature in Canvas to see relevant files:
 - Module will be published before lab
 - Quizzes will be available in the afternoon after lab

▼ Lab 2	
	Lab2.pdf
	Lab 2 Tutorial.ipynb
	Lab 2 Student Activity.ipynb
	JES_Water.csv
	Numpy_Python_Cheat_Sheet.pdf

Tip: Good file storage/management is key for this class and coding/research in general! Create an EVE 310 folder on your computer with a separate subfolder for each lab. Store lab handouts, data, and activities here as soon as you download from Canvas.

Lab 2 Learning Objectives

- Working with Packages
- Importing Data
- Comments
- Arithmetic Operators
- Variable Types
- Arrays and Indexing
- Other Tips: Indenting and Workspace Reset
- Lab 2 Activity

Open the Lab 2 Tutorial Jupyter Notebook to follow along!

Working with Python packages

- The `import` Statement

```
import <package_name>
```

Ex: `import numpy`

- You can also import an entire package under an alternate name:

```
import <package_name> as <alt_name>
```

Ex: `import numpy as np`

- Most famous and useful Python packages (that we will use):
NumPy, Pandas, Matplotlib, Seaborn, SciPy, Scikit-Learn

Working with Python packages

- The `import` Statement

```
import <package_name>
```

Ex: `import numpy`

- You can also import an entire package under an alternate name:

```
import <package_name> as <alt_name>
```

Ex: `import numpy as np`

- If you get a “ModuleNotFound” Error, it is because your editor cannot find the package.
 - Solution: Press Command+Shift+P (on mac) or Ctrl+Shift+P (on windows), type “select interpreter” in the box that appears, and select “base” from the dropdown.

Working with Packages

- Packages need to be imported before packages can be utilized

```
1 #Import Packages
2 import numpy as np
```

Name of Package

Alias, or nickname, assigned to that package within the script.

Note: whatever you put after the “as” is what is used to refer to the function.

- Example:

```
a = np.array( [5, 7, 4] )
```

Output

Package
reference

Function

Input

Working with Packages

- Packages need to be imported before packages can be utilized

```
1 #Import Packages
2 import numpy as np
```

Name of Package

Alias, or nickname, assigned to that package within the script.

Note: whatever you put after the “as” is what is used to refer to the function.

- Example:

```
a = np.array([5, 7, 4])
```

Output

Package
reference

Function

Input

- If you spell the function incorrectly, you get a `AttributeError`.

Installing a Python Package

- Packages need to be installed on your computer before you can import them to your script
- If you see the error below, it means the package needs to be installed

```
1 import tsnet
```

ModuleNotFoundError

Traceback (most recent call last)

<ipython-input-1-b0506a624747> in <module>

----> 1 import tsnet

ModuleNotFoundError: No module named 'tsnet'

Installing a Python Package

- Python packages must be installed before using them for the first time
- To install a package, go to the Explorer in VSCode, right click and select “Open in New Integrated Terminal”, and use the following command in the terminal: `pip install <package_name>`

```
(base) C:\Users\Matthew>pip install usgs
Collecting usgs
  Downloading usgs-0.2.7.tar.gz (9.1 kB)
Requirement already satisfied: click>=4.0 in c:\users\matthew\anaconda3\lib\site-packages (from usgs) (2.24.0)
Requirement already satisfied: requests>=2.7.0 in c:\users\matthew\anaconda3\lib\site-packages (from usgs) (2.24.0)
Collecting requests_futures>=0.9.5
  Downloading requests_futures-1.0.0.tar.gz (10 kB)
Requirement already satisfied: urllib3!=1.25.0,!=1.25.1,<1.26, in c:\users\matthew\anaconda3\lib\site-packages (from requests_futures>=0.9.5) (1.25.9)
Requirement already satisfied: idna<3,>=2.5 in c:\users\matthew\anaconda3\lib\site-packages (from requests_futures>=0.9.5) (2.10)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\matthew\anaconda3\lib\site-packages (from requests_futures>=0.9.5) (2024.7.4)
Requirement already satisfied: chardet<4,>=3.0.2 in c:\users\matthew\anaconda3\lib\site-packages (from requests_futures>=0.9.5) (3.0.4)
Building wheels for collected packages: usgs, requests_futures
  Building wheel for usgs (setup.py) ... done
  Created wheel for usgs: filename=usgs-0.2.7-py3-none-any.whl size=10320 sha256=ee3b998d0e071eb7a893b0d16dc015df55b7dd6f7eb3a41ad0e499f268e0ea8f
  Stored in directory: c:\users\matthew\appdata\local\pip\cache\wheels\07\8a\92\9f1606668be9142d08cfe4b58f12bbbad61b2044c4e698a03
  Building wheel for requests_futures (setup.py) ... done
  Created wheel for requests_futures: filename=requests_futures-1.0.0-py3-none-any.whl size=7016 sha256=5208f81f245ca441c89506fe4534f5fd3d925c1ad5c86bb9dd30d733b74c47f2
  Stored in directory: c:\users\matthew\appdata\local\pip\cache\wheels\ae\8d\24\ab782da8390bb118f413a4a6cb10e10802f9323fa2ea3b1698
Successfully built usgs requests_futures
Installing collected packages: requests_futures, usgs
Successfully installed requests_futures-1.0.0 usgs-0.2.7

(base) C:\Users\Matthew>
```

Step 1: Type “pip install usgs” (to import the usgs package)

Step 2: Wait for the necessary files to download (shouldn't take too long)

Step 3: When you are able to type again into the Command Prompt/Terminal, you know you're done installing

Difference between Installing and Importing

- **Install:**

- Downloading the files on to your computer
- Use `pip install <package name>` in the command prompt
- Only needs to be done once for each package

- **Import:**

- Making the package available for use in a script or notebook
- Use `import` on python script/notebook
- Needs to be done each time you use the package in a script or notebook



Install: Buy the pasta maker from the store

Import: Put the pasta maker on the Kitchen Aid each time you use it

Comments in your Code

`#` = single line comment

`""" """` = multi line comments

- Leave notes to yourself and others in your code
- Any line that starts with `#` will be skipped by the computer, but humans can read it
- Using `""" """`

```
# Can you leave a comment next to the import statements saying what you have imported?  
import numpy as np # I just imported numpy and gave it the alias np  
import pandas as pd  
import matplotlib.pyplot as plt
```

Generate Code Markdown

Variables: Different Types for Different Purposes

- Numbers
- Strings
- List
- Array
- Dataframe

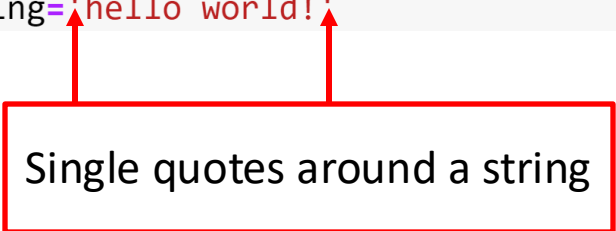
Individual Variables

- **Numbers:** stores a single number

```
4 #Creating a variable b and assigning it the value of 2
5 b = 2
```

- **Strings:** stores a sequence of characters as text

```
12 #Creating a variable mystring and assigning it the value of 'hello world!'
13 mystring='hello world!'
```



Single quotes around a string

- See Links for more info and tutorials:
- Numbers and strings:
https://www.learnPython.org/en/Variables_and_Types

Arithmetic operators

Assume variable **a** holds the value 10 and variable **b** holds the value 2, then:

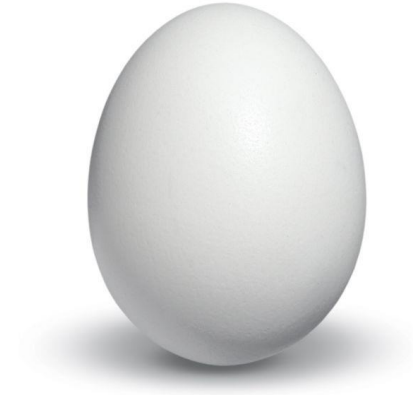
Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 12$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = 8$
* Multiplication	Multiplies values on either side of the operator	$a * b = 20$
/ Division	Divides left hand operand by right hand operand	$a / b = 5$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 100$

Variable Types: Lists, Arrays, Dataframes:

- **List:** Multiple numbers or strings
- **Arrays:** Multiple numbers, commonly used for mathematical operations (NumPy package)
- **Dataframes:** Similar to an array, with more functionality (Pandas package. Note: will explore dataframes more in depth during Lab 3)
- Lists: <https://www.datacamp.com/community/tutorials/lists-Python>
- Arrays: <https://www.datacamp.com/community/tutorials/Python-NumPy-tutorial>
- Dataframes: <https://www.datacamp.com/community/tutorials/Pandas-tutorial-dataframe-Python>

Standard Variable Types Summary

- Numbers
 - Strings
- Individual Elements
(Individual eggs)



- List
 - Array
 - Dataframe
- Groups of Elements
(Egg cartons)



Creating a List

Diagram illustrating the creation of a list in Python:

```
1 #Creating a list
2 my_list = [5, 'word', 74, 'another word']
3
4 print(my_list)
```

Annotations:

- Name of the variable: `my_list`
- Square Brackets on either end: `[` and `]`
- Each element of the list separated by a comma: `5, 'word', 74, 'another word'`

Output:

```
[5, 'word', 74, 'another word']
```

Note: lists cannot be used in mathematical operations

Creating a 1D Array (requires NumPy package)

```
1 # Creating an array
2 my_array = np.array([5,7,4])
3 print(my_array)
```

[5 7 4]

my_array = np.array([5,7,4])

Output Package reference Function Input

Indexing 1D Arrays and Lists

- An array or list has multiple elements
- Can use specific elements of the array or list by using an index

Array (or List): A box of eggs

Value: Contents of the egg

Index: Position of the egg in the box

Important! Python (unlike MATLAB) indexes from 0. Meaning that at the first position, the index = 0

Value:

Index:



Value:
Index:



Indexing 1D Arrays and Lists

Indexing one element:

```
my_array = np.array([5, 7, 4])  
first_val = my_array[0]  
print(first_val)
```

array

index

5

value

Value:
Index:



Indexing all elements in a range (ex: the first two values in the array):

```
first_two_vals = my_array[0:1]  
print(first_two_vals)
```

X [5] ← only give the first value, because indexing is not inclusive of end value!

```
first_two_vals = my_array[0:2]  
print(first_two_vals)
```

[5 7] ← now correctly gives the first two values

When selecting a range of elements, indexing is *not inclusive*, meaning it does not include the value associated with the end index.

Indexing 2D Arrays and Lists

Creating a 2D Array:

```
1 # Creating a 2d array
2 my_array_2D=np.array([[5,7,4],[8,9,2]])
3 print(my_array_2D)
```

First row Second row

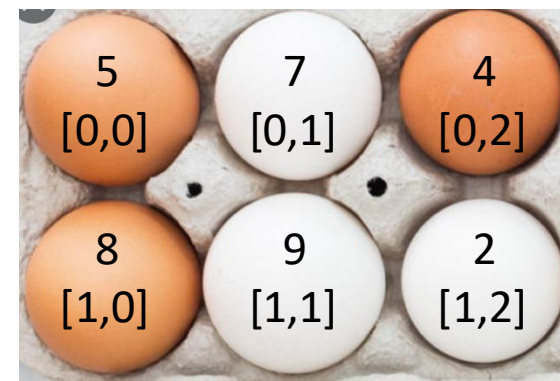
```
[[5 7 4]
 [8 9 2]]
```

Representation of Matrix

Columns →		0	1	2
Rows ↓	0	a_{00}	a_{01}	a_{02}
	1	a_{10}	a_{11}	a_{12}
	2	a_{20}	a_{21}	a_{22}

Value:

Index:



Value:

Index:

Indexing 2D Arrays and Lists

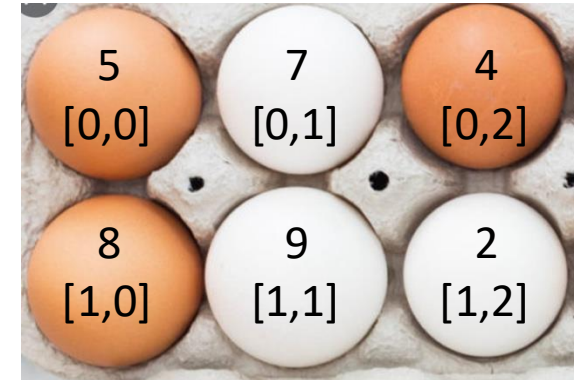
Creating a 2D Array:

```
1 # Creating a 2d array
2 my_array_2D=np.array([[5,7,4],[8,9,2]])
3 print(my_array_2D)
```

First row Second row

[[5 7 4]
[8 9 2]]

Value:
Index:



Value:
Index:

Indexing a 2D Array:

```
6 # Accessing the first row and second column of my_array
7 first_row_second_col=my_array_2D[0,1]
```

Row index Column index

```
16 # Accessing the second row and all columns of my_array
17 second_row_all_col=my_array_2D[1,:]
```

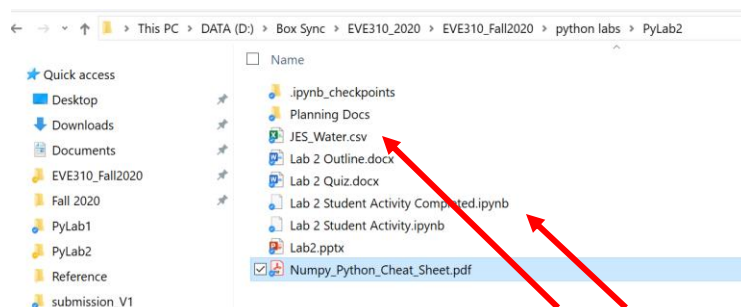
Colon means "all"

Basic Computation with Arrays

- Use **NumPy** functions to do basic computations with arrays
- Examples of operations are: sum, max, min, and standard deviation
- To use these functions, follow this format:
 - `np.function_name(array)` where the `function_name` is the name of the function and `array` is the variable name of the array
- For example, if finding the minimum of the array called `my_array`, use:
 - `np.min(my_array)`
- Other operations (same format):
 - Maximum: `np.max(my_array)`
 - Standard deviation: `np.std(my_array)`

Importing Data

- Python can import data from a file to be turned into a Python variable
- For our purposes, the data file must be saved in the same folder as your Python script or notebook (see below)
- Importing data uses the Pandas package and imports data to Python as a dataframe
- (more on dataframes next time!)



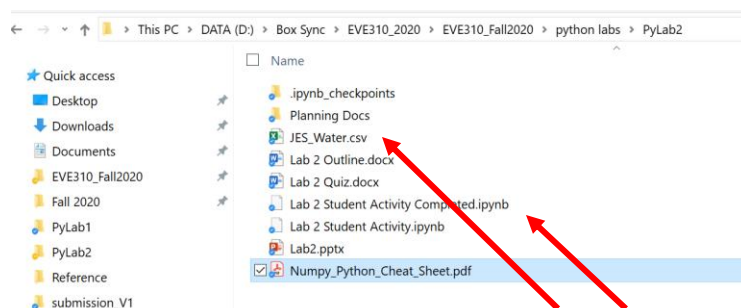
Data and
notebook (or
script) in the
same folder

```
import pandas as pd
water_df = pd.read_csv(< Paste your address here >)
water_df.head(10)
```

Tip: Good file storage/management is key for this class and coding/research in general! Create an EVE 310 folder on your computer with a separate subfolder for each lab. Store lab handouts, data, and activities here as soon as you download from Canvas.

Importing Data

- Python can import data from a file to be turned into a Python variable
- For our purposes, the data file must be saved in the same folder as your Python script or notebook (see below)
- Importing data uses the Pandas package and imports data to Python as a dataframe
- (more on dataframes next time!)



Data and
notebook (or
script) in the
same folder

```
import pandas as pd
water_df = pd.read_csv('/Users/pg13/Documents/Personal-Projects/Labs_2024/Lab2/JES_Water.csv')
water_df.head(10)
```

0.5s

	allDate	prevDay	prevWeek	avgTemp	wF	MonMed	classSession	dayType	season
0	1/14/2009 0:00	NaN	NaN	47.0	7.192282	10.788424	0	wk	Winter
1	1/15/2009 0:00	7.192282	NaN	46.0	6.813741	10.788424	0	wk	Winter
2	1/16/2009 0:00	6.813741	NaN	41.5	7.570824	10.788424	0	wk	Winter
3	1/17/2009 0:00	7.570824	NaN	57.5	5.678118	10.788424	0	wknd	Winter
4	1/18/2009 0:00	5.678118	NaN	61.0	6.435200	10.788424	0	wknd	Winter
5	1/19/2009 0:00	6.435200	NaN	58.0	9.084988	10.788424	0	wk	Winter
6	1/20/2009 0:00	9.084988	NaN	50.0	14.384565	10.788424	1	wk	Winter
7	1/21/2009 0:00	14.384565	7.192282	53.5	15.141647	10.788424	1	wk	Winter
8	1/22/2009 0:00	15.141647	6.813741	61.0	14.006024	10.788424	1	wk	Winter
9	1/23/2009 0:00	14.006024	7.570824	68.0	12.491859	10.788424	1	wk	Winter

Tip: Good file storage/management is key for this class and coding/research in general! Create an EVE 310 folder on your computer with a separate subfolder for each lab. Store lab handouts, data, and activities here as soon as you download from Canvas.

Let's talk about NaN!

8/28/12 0:00	138.091822	104.174532	87.5	136.312679	102.736076	1 wk	Summer
8/29/12 0:00	136.312679	102.736076	88.5	NaN	102.736076	1 wk	Summer
8/30/12 0:00	NaN	97.3607912	90.5	NaN	102.736076	1 wk	Summer
8/31/12 0:00	NaN	103.606721	86.5	168.488679	102.736076	1 wk	Summer
9/1/12 0:00	168.488679	82.9383723	89.5	79.4936476	116.249996	1 wknd	Fall
9/2/12 0:00	79.4936476	111.972481	87.5	99.5941842	116.249996	1 wknd	Fall
9/3/12 0:00	99.5941842	138.091822	88	132.943662	116.249996	1 wk	Fall

- NaN: very common in real world data
- `np.min` and `np.max` do not work with arrays with NaN values
- `min()` and `max()` do not work if array starts with NaN values
- **Instead, use:**
 - `np.nanmin()` and `np.nanmax()` are the most robust functions to handle NaNs
- If we use `np.min` and `np.max` with this data, our answers will be incorrect!
- Always inspect your data and spot check your work

```
In [8]: np_min = np.min(water_con)
         np_max = np.max(water_con)

         py_min = min(water_con)
         py_max = max(water_con)

         np_nanmin = np.nanmin(water_con)
         np_nanmax = np.nanmax(water_con)

         print('np.min:', np_min)
         print('np.max:', np_max)

         print('py min:', py_min)
         print('py max:', py_max)

         print('np.nanmin:', np_nanmin)
         print('np.nanmax:', np_nanmax)
```

Incorrect → `np.min: 2.536225899`
Correct → `np.max: 148.6909751`
Correct → `py min: 0.037854118`
 → `py max: 188.2106742`
 → `np.nanmin: 0.037854118`
 → `np.nanmax: 188.2106742`

More to come on data cleaning this semester!

Indenting

- Except for specific cases, Python expects there to be no indents or spaces when beginning a line of code

There is a
space there

```
1 # A simple example of code that works
2 a=1
3
4 # A simple example of code that does NOT work
5  a=1
6
7
```

File "<ipython-input-82-db1f3e6bddbb>", line 5
a=1
^
IndentationError: unexpected indent

ERRORS IN PYTHON

Errors	Description
SyntaxError	Occurs when the code violates Python's syntax rules.
NameError	Occurs when you attempt to use or access a variable, function, or module name that has not been defined yet or is out of scope.
IndentationError	Occurs when the spaces at the beginning of a code line do not follow expected patterns.
TypeError	Occurs when an operation is performed or a function is applied on an object of an inappropriate type.
ValueError	Raised when a built-in operation or function receives an argument that has the right type but an inappropriate value.
KeyError	Occurs when a program tries to access a key in a dictionary that doesn't exist.
IndexError	Occurs when you try to access an element in a list, tuple, or any other sequence using an invalid index.
ZeroDivisionError	Raised when you attempt to divide a number by zero.
FloatingPointError	Occur when working with floating-point numbers.
ModuleNotFoundError	Raised when a module could not be found.
ImportError	Raised when an import statement fails to import a module or name within the code.

Resetting the Workspace

- Sometimes you will want to run your code again and clear all existing variables in the workspace.
- In a Jupyter Notebook, click the “ Restart” button on the top

Look at the NumPy Cheat Sheet on Canvas

Python For Data Science Cheat Sheet

NumPy Basics

Learn Python for Data Science Interactively at www.DataCamp.com



NumPy

The NumPy library is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Use the following import convention:

```
>>> import numpy as np
```



NumPy Arrays

1D array

```
1 2 3
```

2D array

axis 1
axis 0

```
1.5 2. 3.  
4. 5. 6.
```

3D array

axis 2
axis 1
axis 0

Creating Arrays

```
>>> a = np.array([1,2,3])  
>>> b = np.array([(1.5,2,3), (4,5,6)], dtype = float)  
>>> c = np.array([(1.5,2,3), (4,5,6)], [(3,2,1), (4,5,6)]],  
                dtype = float)
```

Initial Placeholders

```
>>> np.zeros((3,4))  
>>> np.ones((2,3,4),dtype=np.int16)  
>>> d = np.arange(10,25,5)  
  
>>> np.linspace(0,2,9)  
  
>>> e = np.full((2,2),7)  
>>> f = np.eye(2)  
>>> np.random.random((2,2))  
>>> np.empty((3,2))
```

Create an array of zeros
Create an array of ones
Create an array of evenly spaced values (step value)
Create an array of evenly spaced values (number of samples)
Create a constant array
Create a 2X2 identity matrix
Create an array with random values
Create an empty array

I/O

Saving & Loading On Disk

```
>>> np.save('my_array', a)  
>>> np savez('array.npz', a, b)  
>>> np.load('my_array.npy')
```

Saving & Loading Text Files

```
>>> np.loadtxt('myfile.txt')  
>>> np.genfromtxt('my_file.csv', delimiter=',')  
>>> np.savetxt('myarray.txt', a, delimiter=" ")
```

Data Types

```
>>> np.int64  
>>> np.float32  
>>> np.complex  
>>> np.bool  
>>> np.object  
>>> np.string_  
>>> np.unicode_
```

Signed 64-bit integer types
Standard double-precision floating point
Complex numbers represented by 128 floats
Boolean type storing TRUE and FALSE values
Python object type
Fixed-length string type
Fixed-length unicode type

Inspecting Your Array

```
>>> a.shape  
>>> len(a)  
>>> b.ndim  
>>> e.size  
>>> b.dtype  
>>> b.dtype.name  
>>> b.astype(int)
```

Array dimensions
Length of array
Number of array dimensions
Number of array elements
Data type of array elements
Name of data type
Convert an array to a different type

Asking For Help

```
>>> np.info(np.ndarray.dtype)
```

Array Mathematics

Arithmetic Operations

```
>>> g = a - b  
array([[ -0.5,  0. ,  0. ],  
       [ -3. , -3. , -3. ]])  
>>> np.subtract(a,b)  
>>> b + a  
array([[ 2.5,  4. ,  6. ],  
       [ 5. ,  7. ,  9. ]])  
>>> np.add(b,a)  
>>> a / b  
array([[ 0.66666667,  1. ,  1. ],  
       [ 0.25 ,  0.4 ,  0.5 ]])  
>>> np.divide(a,b)  
>>> a * b  
array([[ 1.5,  4. ,  9. ],  
       [ 4. , 10. , 18. ]])  
>>> np.multiply(a,b)  
>>> np.exp(b)  
>>> np.sin(a)  
>>> np.cos(b)  
>>> np.log(a)  
>>> e.dot(f)  
array([[ 7. ,  7. ],  
       [ 7. ,  7. ]])
```

Subtraction
Subtraction
Addition
Addition
Division
Division
Multiplication
Multiplication
Exponentiation
Square root
Print sines of an array
Element-wise cosine
Element-wise natural logarithm
Dot product

Comparison

```
>>> a == b  
array([[False,  True,  True],  
       [False, False, False]], dtype=bool)  
>>> a < 2  
array([[ True, False, False],  
       [ True,  True,  True]], dtype=bool)  
>>> np.array_equal(a, b)
```

Element-wise comparison
Element-wise comparison
Array-wise comparison

Aggregate Functions

```
>>> a.sum()  
>>> a.min()  
>>> b.max(axis=0)  
>>> b.cumsum(axis=1)  
>>> a.mean()  
>>> b.median()  
>>> a.corrcoef()  
>>> np.std(b)
```

Array-wise sum
Array-wise minimum value
Maximum value of an array row
Cumulative sum of the elements
Mean
Median
Correlation coefficient
Standard deviation

Copying Arrays

```
>>> h = a.view()  
>>> np.copy(a)  
>>> h = a.copy()
```

Create a view of the array with the same data
Create a copy of the array
Create a deep copy of the array

Sorting Arrays

```
>>> a.sort()  
>>> c.sort(axis=0)
```

Sort an array
Sort the elements of an array's axis

Subsetting, Slicing, Indexing

Also see Lists

```
Subsetting  
>>> a[2]  
3  
>>> b[1,2]  
6.0  
  
Slicing  
>>> a[0:2]  
array([1, 2])  
>>> b[0:2,1]  
array([ 2.,  5.])  
  
>>> b[:1]  
array([[1.5, 2., 3.]])  
>>> c[1,...]  
array([[ 3.,  2.,  1.],  
       [ 4.,  5.,  6.]])  
  
>>> a[ : :-1]  
array([3, 2, 1])  
  
Boolean Indexing  
>>> a[a<2]  
array([1])  
  
Fancy Indexing  
>>> b[[1, 0, 1, 0], [0, 1, 2, 0]]  
array([ 4. ,  2. ,  6. , 1.5])  
>>> b[[1, 0, 1, 0], :][:, [0,1,2,0]]  
array([[ 4. ,  2. ,  6. ,  4. ],  
       [ 1.5,  2. ,  3. , 1.5],  
       [ 4. ,  2. ,  6. ,  4. ],  
       [ 1.5,  2. ,  3. , 1.5]])
```

Select the element at the 2nd index
Select the element at row 1 column 2 (equivalent to b[1][2])
Select items at index 0 and 1
Select items at rows 0 and 1 in column 1
Select all items at row 0 (equivalent to b[0:1, :])
Same as [1, :, :]
Reversed array a
Select elements from a less than 2
Select elements (1,0), (0,1), (1,2) and (0,0) (equivalent to b[0:1, :])
Select a subset of the matrix's rows and columns

Array Manipulation

```
Transposing Array  
>>> i = np.transpose(b)  
>>> i.T  
  
Changing Array Shape  
>>> b.ravel()  
>>> g.reshape(3,-2)  
  
Adding/Removing Elements  
>>> h.resize((2,6))  
>>> np.append(h,g)  
>>> np.insert(a, 1, 5)  
>>> np.delete(a, [1])  
  
Combining Arrays  
>>> np.concatenate((a,d),axis=0)  
array([ 1,  2,  3, 10, 15, 20])  
>>> np.vstack((a,b))  
array([[ 1. ,  2. ,  3. ],  
       [ 1.5,  2. ,  3. ],  
       [ 4. ,  5. ,  6. ]])  
  
>>> np.r_[e,f]  
>>> np.hstack((e,f))  
array([[ 7. ,  7. ,  1. ,  0. ],  
       [ 7. ,  7. ,  0. ,  1.]])  
>>> np.column_stack((a,d))  
array([[ 1, 10],  
       [ 2, 15],  
       [ 3, 20]])  
  
>>> np.c_[a,d]  
  
Splitting Arrays  
>>> np.hsplit(a,3)  
[array([1]), array([2]), array([3])]   
>>> np.vsplit(c,2)  
[array([[ 1.5,  2. ,  1. ],  
       [ 4. ,  5. ,  6. ]]),  
 array([[ 3. ,  2. ,  3. ],  
       [ 4. ,  5. ,  6. ]])]
```

Permute array dimensions
Permute array dimensions
Flatten the array
Reshape, but don't change data
Return a new array with shape (2,6)
Append items to an array
Insert items in an array
Delete items from an array
Concatenate arrays
Stack arrays vertically (row-wise)
Stack arrays vertically (row-wise)
Stack arrays horizontally (column-wise)
Create stacked column-wise arrays
Create stacked column-wise arrays
Split the array horizontally at the 3rd index
Split the array vertically at the 2nd index

DataCamp
Learn Python for Data Science Interactively



In Class Activity

- Work through Lab 2 Student Activity notebook
- **Make sure you can complete the activity before taking the Lab 2 quiz. The quiz will ask you some questions about the activity and code outputs.**
- Self-Assessment (ungraded) on Canvas
- Lab 2 Quiz due Friday, September 5, 11:59 pm

Recap

- Working with Packages
- Importing Data
- Comments
- Arithmetic Operators
- Variable Types
- Arrays and Indexing
- Other Tips: Indenting and Workspace Reset
- Lab 2 Activity